

Semantic code search, built as an argument you can defend line by line

CodeSeek indexes a real GitHub repository — every function, class, notebook cell, and doc page — with structure-aware chunking, then answers plain-English questions with an LLM agent that searches, follows references, reads source, and cites `file:line` locations mechanically checked against the real repo before you ever see them. Written directly from the current `src/codeseek` source tree, its docs, and its CI config — every number below traces to a line in the repository, not to marketing copy about it.

1536 DIMS / CHUNK (OPENAI)	400 MAX TOKENS / CHUNK	1.000 MRR AFTER RRF FUSION
0.803 MRR, RERANK ALONE	8 MAX AGENT ITERATIONS	44/44 FAST TESTS PASSING

HOW THIS PROJECT ACTUALLY EVOLVED — TELL IT AS A STORY

It didn't start as an agent. It started narrower and cheaper: does chunking code by real AST structure, and comparing a general-purpose vs. code-specialized embedding model, actually change what gets retrieved — measured, not assumed. That ran on two local `sentence-transformers` models, \$0 API cost. Once the project grew an actual answering step (the `/explain` agent loop), it needed a real LLM provider. OpenAI has no code-specialized embedding model, so pairing two generic OpenAI embeddings under "general"/"code" labels would've been a fake comparison — so embeddings consolidated to *one* model (`text-embedding-3-small`), and the interesting comparison moved from "which embedding model" to "which retrieval stage actually helps" (hybrid → rerank → RRF fusion) — a more production-shaped question.

CONTENTS

- | | |
|--|--------------------------|
| 01 Architecture at a glance | 14 Citation verification |
| 02 Ingestion | 15 Eval harness |
| 03 Chunking: the shared packer | 16 CI quality gate |
| 04 Chunking: Python (ast) | 17 Measured results |
| 05 Chunking: JS/TS (tree-sitter) | 18 Pipeline wiring |
| 06 Chunking: notebook / docs / generic | 19 API & background jobs |
| 07 Embedding | 20 Observability |
| 08 Schema & Qdrant | 21 The Streamlit UI |

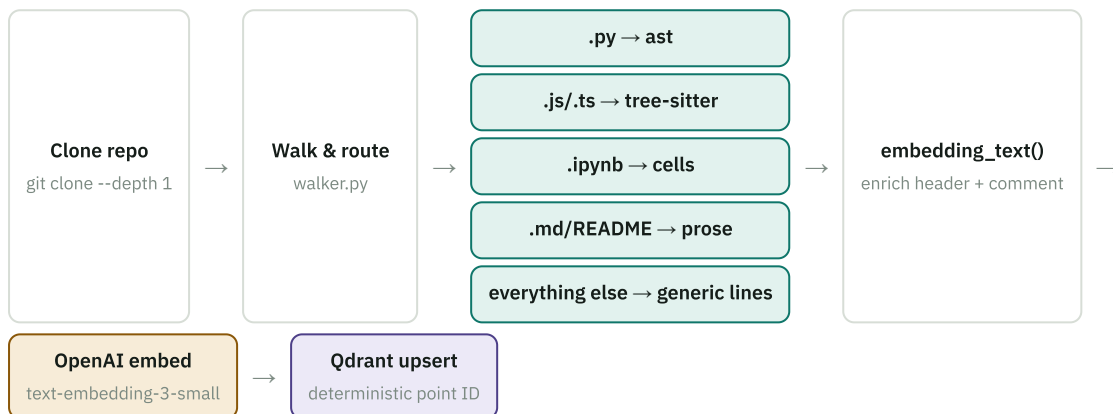
- 09 Hybrid retrieval
- 10 Rerank + RRF fusion
- 11 HyDE (a negative result)
- 12 The explain loop
- 13 Tools
- 22 Testing strategy
- 23 Scaling design
- 24 Q&A cheat sheet
- 25 Glossary

01 – the whole system in one picture

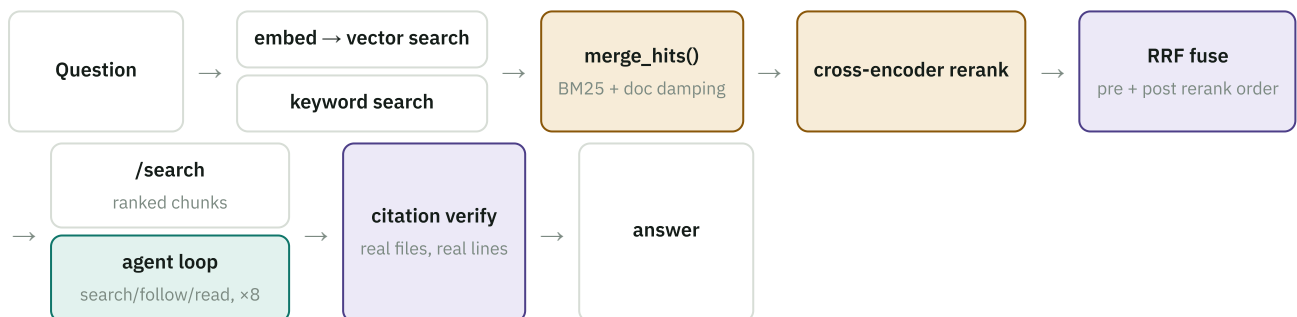
Two entry points, one indexed corpus

Everything downstream reduces to two paths: build the index once, then answer questions against it two ways.

indexing path – POST /ingest (background job)



query path – POST /search or POST /explain



IF THEY ASK "WALK ME THROUGH YOUR ARCHITECTURE"

Say the two-path sentence above, then pick *one* box and go one level deeper. Reciting all sixteen boxes reads as memorization; zooming in on demand reads as understanding.

Ingestion: clone → walk → route

ingestion/clone.py

ingestion/walker.py

ingestion/manifest.py

config.py

What happens. `clone_or_update()` does a shallow `git clone --depth 1` (or a shallow `pull` if the repo already exists) — a full-history clone is pure waste when only the current tree gets indexed. Git failures get 3 retries with exponential backoff via `tenacity`, because a bad URL and a dropped connection both raise identical exceptions and telling them apart isn't worth parsing git's `stderr`.

Routing. `walker.py` classifies every file into exactly one bucket — `code` (has a real parser: `.py`, `.js/.jsx/.mjs/.cjs`, `.ts/.tsx`), `docs` (`README*`, `.md`), `notebook` (`.ipynb`), `code_generic` (~50 more extensions with no parser — Go, Rust, Java, YAML, SQL, Dockerfiles), or dropped. `node_modules`, `.git`, `__pycache__`, binaries, and lockfiles are skipped outright.

WHY A DENYLIST FOR SKIP, NOT AN ALLOWLIST FOR EVERYTHING

Ingestion is deliberately permissive: an unrecognized extension falls through to `code_generic` (real, citeable line-range chunks) rather than being dropped. This is what makes "index almost any repo" actually true — a Rust or Go repo isn't a second-class citizen, it just gets line-based instead of symbol-level chunks.

One more routing rule: `_is_translated_doc()` skips non-English locale subfolders under any `docs/` directory. A repo whose docs are translated into ~20 languages would otherwise have indexing cost dominated by redundant translated copies of the same content.

WHAT'S PASSED DOWNSTREAM

A stream of `FileRecord`s (repo, language, category, POSIX-normalized path, size) — nothing is embedded yet. Paths are normalized to forward slashes with `.as_posix()` at the source, not at every comparison site downstream — a Windows/Linux path-separator mismatch anywhere later would otherwise silently break every ground-truth string comparison in the eval harness.

Chunking: one shared packing algorithm, five chunkers

chunking/shared.py

This is the idea the whole project exists to test: a function is a unit of meaning, and a fixed-size character window cuts it in half roughly as often as not — a wrong answer, not an approximate one.

Every chunker calls the same primitive: `pack_units(units, max_tokens, overlap_ratio, joiner)`. It greedily packs atomic units — lines for code, sentences for prose — into chunks capped at **400 tokens** (~300 words, counted with `tiktoken`'s real `cl100k_base` encoding, not a word-count guess). Only when a single unit itself exceeds the cap does it fall back to a last-resort word-boundary split — never mid-token.

```
for idx, unit in enumerate(units):
    # flush the current chunk before it would exceed max_tokens,
    # keeping overlap_ratio of it for continuity into the next one
    prospective = current + [idx]
    if current and count_tokens(joiner.join(...)) > max_tokens:
        flush()
    current.append(idx)
```

WHY ONE SHARED FUNCTION INSTEAD OF FIVE COPIES

The oversized-split logic (overlap ratio, hard word-boundary fallback, index bookkeeping so callers can map a piece back to real line numbers) is genuinely tricky to get right, and every chunker needs it identically. Factoring it once means a fix or a tuning change lands everywhere at once instead of five synchronized edits.

Python chunking: real AST, method-level granularity

chunking/code_chunker.py

Python ships a parser for free (`ast.parse`), producing real structural nodes — `FunctionDef`, `ClassDef`, `AsyncFunctionDef`. The chunker walks only *top-level* children, so a function stays a function and a class becomes: one **overview chunk** (header, docstring, field declarations, every method's *signature line only*) plus one **dedicated chunk per method**, each individually retrievable and citeable.

WHY SPLIT A CLASS INTO OVERVIEW + PER-METHOD CHUNKS

A whole-class chunk was measured as the single biggest ceiling on retrieval quality: a 15-method class became one chunk (or an arbitrary token-sliced piece that could cut a method in half), so a question about one method could never retrieve just that method. It also fixes a real failure mode: a dataclass-style class with no docstring (typer's `OptionInfo`) embeds almost no semantic signal as a raw body dump — but its field declarations (in the overview) and its `__init__` (its own chunk) both carry the meaning a natural-language question is looking for.

Decorators are pulled into the chunk's line span (so `@app.get(...)` stays attached), and `_leading_comment()` walks backward collecting a contiguous `#`-comment block above a node — because `ast` attaches a docstring automatically but a comment *above* a `def` is invisible to the tree entirely.

WHAT'S PASSED DOWNSTREAM

A list of `CodeChunk` (repo, language, path, symbol_name, symbol_type, start/end line, text, leading_comment). An oversized function splits into `FunctionName#part1`, `#part2` ... — a convention that reappears in the eval harness and `find_by_symbol` because it must count as "still a match," not a miss.

JS/TS chunking: tree-sitter fills the gap Python's ast has for free

chunking/js_ts_chunker.py

JavaScript/TypeScript have no parser Python can call natively, so this uses **tree-sitter** — the same incremental-parsing engine GitHub uses for code navigation — via `tree-sitter-javascript` / `tree-sitter-typescript`. It matches four top-level shapes: `function foo(){} ,` `class Foo{} ,` the common `const foo = () => {}` arrow pattern, and any wrapped in `export / export default`. Everything else — a bare `export {a, b}`, malformed source — is silently skipped; tree-sitter never throws on bad input, it marks the broken node and keeps going.

WHY SWAP THE ORIGINALLY-PLANNED TREE-SITTER PACKAGE

The first choice, `tree-sitter-languages`, called an old API signature that no longer exists in the installed version — found while researching the approach, before writing the chunker itself. Verifying a dependency works before building on it turned this into a five-minute research finding instead of a broken chunker discovered mid-build.

Its JSDoc/comment extractor mirrors Python's leading-comment logic, additionally handling a `/* ... */` block scanning backward for the opening `/*` — tree-sitter's span for a function starts at the keyword itself, and a comment above it is always a separate sibling node.

Notebook, docs, and generic chunkers

`chunking/notebook_chunker.py``chunking/docs_chunker.py``chunking/generic_chunker.py`

Notebooks parse as JSON, one chunk per cell (code or markdown — execution outputs, often huge base64 image blobs, are dropped entirely). No stable per-cell line numbers exist, so these chunks carry `start_line=end_line=0` and address by cell position (`cell[3]:code`) — the citation verifier has an explicit rule for exactly this case.

Docs split on sentence boundaries into ~400-token chunks with 15% overlap, so a fact on a chunk boundary still appears whole in a neighbor. `_strip_badge_and_html_noise()` removes CI/coverage badges and raw HTML first — found by diagnosing why a generic opening paragraph kept outranking the actually-relevant implementation in eval results.

Generic code (Go, Rust, Java, YAML, SQL, Dockerfiles, ~50 extensions) falls back to plain line-based packing — real line ranges, addressed by position (`block#4`) since there's no parser to name a symbol. This is the difference between "index almost any file" and "only Python and JS."

Embedding: enriching text before it ever reaches the model

[embedding/context.py](#)
[embedding/openai_embedder.py](#)
[embedding/service.py](#)
[embedding/registry.py](#)

An embedding is a few hundred (here, **1536**) numbers representing meaning — similar meaning lands close together. Every chunk is embedded once, by **text-embedding-3-small** (OpenAI), the single model this project indexes and searches with.

What actually gets embedded isn't the raw chunk text. `embedding_text()` builds a separate string — a header of `{symbol_type} {symbol_name} ({split identifier}) in {path}`, plus the leading comment, prepended ahead of source. `split_identifier()` turns `ArgumentInfo` into "Argument Info."

WHY SPLIT THE IDENTIFIER INTO WORDS

Found via a real eval failure: a dataclass-style class is almost entirely typed field declarations with no prose — its raw-text embedding carries little semantic signal. The identifier itself *is* the answer ("ArgumentInfo" answers "how does typer represent an argument") — but only split into words a general-purpose embedder can match against question phrasing.

`EmbeddingService.embed_one()` caches repeated identical queries (LRU, default 256) — a measured **15× speedup** (21.2ms → 1.4ms) on a cache hit. Retries scope narrowly to transient errors — a 400/401 fails identically on every retry, so retrying it just delays surfacing a real problem.

PRODUCTION-SHAPED DECISION

The `Embedder` interface is a `Protocol`, not a base class — this is what lets tests hand the whole pipeline a `FakeEmbedder` with zero network calls, and it's why the test suite runs in ~10 seconds instead of downloading models.

Schema & Qdrant: idempotency by construction

store/schema.py

store/qdrant_store.py

Qdrant answers "which of these million number-lists is closest to this one," fast, via **HNSW** (a navigable graph that skips most of the index instead of scanning every point). Every chunk becomes one Qdrant point: a vector plus a `ChunkPayload` (repo, language, path, symbol name/type, line range, text, license, stars).

THE SINGLE MOST LOAD-BEARING LINE IN THE STORAGE LAYER

`ChunkPayload.point_id()` derives a deterministic `uuid5` from `repo:path:symbol_name:start_line:end_line` — not a random UUID. Re-indexing an unchanged chunk computes the exact same ID and *overwrites* the same point instead of duplicating. This one decision is what makes incremental re-indexing safe with no separate dedup logic.

`ensure_collection()` creates the vector collection plus two payload indexes (`text`, `symbol_name`), which is what makes `keyword_search()` a real full-text match. `find_by_symbol()` is the "go to definition" primitive: exact-or-split-part-or-method match within one repo.

THREE STORAGE MODES, ONE INTERFACE

`QdrantStore` takes `path=` (on-disk, dev/prod default), `url=` (real containerized server, used for load testing), or `location=":memory:"` (tests). On-disk mode does *not* build a real HNSW index — exactly why the load test downloaded the actual server binary instead of trusting local-mode numbers.

Hybrid retrieval: vector + keyword, merged additively

retrieval/hybrid.py

Pure vector search has a real weakness: a short, exact query like a literal identifier (`JWTBearer`) doesn't embed distinctively enough to reliably win on cosine similarity. So every search runs vector search *and* keyword match in parallel over an over-fetched pool (`fetch_k_for(top_k) = max(top_k*3, 30)`), then merges.

A keyword-only hit doesn't get a flat placeholder score — `_bm25_relevance_for_keyword_only()` fits a real `BM25kapi` index fresh over the fetched pool per query, scores by genuine TF-IDF relevance, and scales that into `[floor, ceiling]` of the vector pool's range. A rare exact term ranks higher within that band, but the ceiling is still the vector pool's own floor — a keyword hit can never outrank a genuinely strong semantic match.

THE IDENTIFIER SHORT-CIRCUIT

If the query looks like a bare dotted identifier and a repo is specified, `find_by_symbol()` runs first — a deterministic "go to definition" match, not a similarity guess that might rank a similarly-named-but-wrong chunk above the real one. Falls through to the normal pipeline if nothing matches.

DOC-OVERVIEW DAMPING — NARROW ON PURPOSE

A README's first chunk (`doc#0`) is broad enough to score deceptively high against almost any question about that repo. The fix damps *only* that chunk (score $\times 0.85$) — measured: damping every doc chunk helped code questions but hurt doc-appropriate ones; damping just `doc#0` improved *both* (code MRR 0.250→0.312, doc MRR 0.167→0.250).

Cross-encoder rerank, corrected by RRF fusion

`retrieval/reranker.py``retrieval/hybrid.py (rrf_fuse)`

First-stage search is a **bi-encoder**: query and chunk embedded independently, compared afterward. A **cross-encoder** (`ms-marco-MiniLM-L-6-v2`) feeds `(query, chunk)` through one model *together* — more accurate, at the cost of one forward pass per candidate, which is why it only runs on the narrow ~30-candidate pool.

THE FAILURE THIS PROJECT MEASURED, NOT ASSUMED

The reranker is trained on prose ranking (MS MARCO) — biased toward README-shaped text. On this codebase's own ablation, adding rerank *alone* on top of hybrid retrieval **dropped MRR from 1.000 to 0.803**. On a live query, it took a target class ranked #2 of 30 and buried it at #23 under several README chunks.

THE FIX, ALSO MEASURED

Reciprocal Rank Fusion (`RRF_K = 60`) blends the reranker's opinion with the pre-rerank order *by rank*, not raw score. $\text{score}(\text{id}) = 1/(60+\text{pre_rank}) + 1/(60+\text{post_rank})$. This recovers MRR to **1.000** — the reranker influences the result without fully overriding a candidate that already scored strongly.

HyDE: implemented, measured, and kept opt-in on purpose

retrieval/hyde.py

HyDE asks a small, cheap model (`gpt-5.4-nano`) to write a hypothetical answer for the query, then embeds *that* instead — meant to close the vocabulary gap between question phrasing and code-shaped answers. Deliberately narrow: only the vector leg changes; the keyword/BM25 leg still sees the raw question.

MEASURED RESULT: IT HURT, HERE

On the 28-question hard-eval set, HyDE dropped MRR from 0.614 to **0.578** — worse than the original baseline (0.608). Plausible reason: these questions ask about this codebase's own idiosyncratic implementation, and a small model's "plausible-sounding" guess drifts *away* from the actual wording rather than toward it.

It stays wired as a real, tested code path — behind an opt-in flag, never the production default. The honest way to keep a plausible technique that didn't pan out: don't delete the evidence, don't ship it as default either.

The explain loop: a tool-calling agent, not a fancier search

agent/loop.py

Pure retrieval has two structural gaps: a one-shot search can't chase a reference across chunks, and a ranked list isn't an *answer*. `explain()` / `explain_stream()` runs an OpenAI tool-calling loop (`gpt-5.4-mini`) iterating up to **8 times**, calling `search_code` / `follow_symbol` / `read_file` until it either has grounded evidence to answer with citations or admits it couldn't find one.

WHY ONE AGENT LOOPING, NOT MULTIPLE AGENTS

Same pattern Claude Code / Codex use: a single model calling tools repeatedly, growing one message history, not multiple cooperating agents. Simpler, cheaper, no demonstrated win from splitting roles at this scale.

The system prompt requires every claim to be backed by a tool call made *that turn*, cited as `repo/path:start-end` ; if the tools don't turn up enough, the model is told to say so rather than answer from general knowledge about what a library "usually" does.

REAL COST ACCOUNTING, NOT A ROUGH ESTIMATE

Each iteration *resends the entire growing message history*. Usage tracks `cached_input_tokens` separately (OpenAI prices a repeated prefix at roughly a 90% discount), so `cost_usd` is the actual bill, not a worst-case ceiling. Pricing constants are dated and sourced against the live pricing page, not left to go stale silently.

The three tools, and why each has a hard cap

agent/tools.py

`build_tools()` is a factory closure bound to one request's repo — not global state — so concurrent `/explain` requests for different repos never cross-contaminate. `search_code` wraps the full hybrid+rerank+RRF pipeline; `follow_symbol` wraps `find_by_symbol`; `read_file` reads raw source, optionally bounded to a line range.

A REAL COST BUG, FOUND AND FIXED

Because every iteration resends the whole history, an uncapped tool result is paid for again on *every subsequent turn*. `follow_symbol` originally had no cap at all; `search_code`'s `top_k` had no ceiling. Both are now hard-capped (`MAX_TOOL_RESULT_CHARS = 8000`, `MAX_SEARCH_TOP_K = 20`) — found via a real cost analysis of a multi-iteration conversation.

READ_FILE IS A PATH-TRAVERSAL BOUNDARY, TREATED AS ONE

`path` is untrusted model output. `read_file` resolves it against the repo root and checks `is_relative_to(repo_root)` before opening it — a model asking for `../../etc/passwd`-shaped paths gets a clean error string, not a real read outside the sandboxed clone.

Citation verification: deterministic, so it can't itself hallucinate

agent/verify.py

`verify_citations()` regex-matches every `path:start-end` in the agent's final answer and checks, in plain Python (no LLM call): does the file exist, is the line range in bounds. This turns "the agent *claims* it's grounded" into "the agent's claims are checked against the real repo" — and because the check is deterministic code, it can't hallucinate the way a second LLM-based verifier could.

DELIBERATELY NARROW — AND THAT'S THE HONEST PART

This confirms a citation's *location is real*; it does not confirm the cited lines support the adjacent claim — that would need semantic judgment (another LLM call), trading away the one property this is built for. A citation pointing at a nonexistent file or out-of-bounds line is the most damaging *and* the most mechanically detectable failure mode.

One special case: `path:0-0` is not treated as broken. Notebook cells and doc chunks store `start_line=end_line=0` by design — bounds-checking 0-0 against a real line count would fail every notebook/doc citation regardless of whether the file is genuinely right.

The eval harness: Recall@k and MRR against hand-verified ground truth

`eval/ground_truth.py`

`eval/harness.py`

`eval/metrics.py`

A `GroundTruthItem` pairs a natural-language question with the exact repo/path/symbol that should answer it.

- **Recall@k** – was the correct chunk anywhere in the top k? Binary per question, averaged.
- **MRR** – $1/\text{rank}$ of the first correct hit (0 if absent). Rewards ranking it near the top.

A DESIGN CHOICE THAT MATTERS MORE THAN IT LOOKS: NO REPO FILTER

`run_eval()` deliberately does not filter search to the ground-truth item's own repo – a real user doesn't tell the system which repo to look in. Narrowing to the known-correct repo would inflate recall relative to what an actual query experiences.

A REAL, SILENT SCORING BUG – GOOD INTERVIEW MATERIAL

An oversized symbol splits into `Name#part1`, `#part2 ...` `_is_relevant()` accepts any part (and any method of a target class) as a match – otherwise every split symbol scores a permanent, silent zero regardless of how good retrieval actually is.

The CI quality gate: retrieval as a tested contract

`scripts/eval_gate.py``.github/workflows/ci.yml`

`eval_gate.py` indexes CodeSeek's own source tree and re-runs the ground-truth set on every push. If Recall@5 drops below 0.90 or MRR below 0.85 — thresholds set under the measured 1.000 baseline, loose enough for legitimate model drift but tight enough to catch a real regression — CI fails the same way a broken unit test would.

CI JOB	WHAT IT CHECKS	RUNS ON
lint-type-test	ruff, mypy, fast pytest (fakes only)	Python 3.10 & 3.12
eval-gate	Recall@5 $\geq$ 0.90, MRR $\geq$ 0.85	after lint-type-test

GRACEFUL DEGRADATION, APPLIED TO CI ITSELF

The gate needs a real `OPENAI_API_KEY` — a fork's PR has no access to repo secrets, so the gate checks for the key and exits 0 with a clear message if absent, rather than failing every external contributor's CI on something structurally impossible for them to provide.

The numbers this project can actually show

Ablation (10 questions, 138 chunks, CodeSeek's own source):

STAGE	RECALL@5	RECALL@10	MRR
1. Vector-only	1.000	1.000	1.000
2. + keyword hybrid	1.000	1.000	1.000
3. + cross-encoder rerank (no fusion)	1.000	1.000	0.803
4. + RRF fusion (production)	1.000	1.000	1.000

Harder eval (28 paraphrased questions) — a real curve, not a ceiling:

LABEL	RECALL@5	RECALL@10	MRR	VERDICT
baseline	0.786	0.929	0.608	—
+ comment/JSDoc enrichment	0.786	0.964	0.623	kept — modest real gain
+ real BM25 keyword scoring	0.750	0.964	0.614	flat-to-negative here
+ HyDE query expansion	0.786	0.893	0.578	hurt — kept opt-in only

Load test (6 real repos, 13,014 chunks total):

THE ACTUAL FINDING, AND IT'S THE INTERESTING ONE

Adding `itsdangerous` (190 chunks, a 4% bump on 4,397 already indexed) still measured ~4 seconds of query latency — statistically indistinguishable from repos holding 30× more data. Latency is dominated by **fixed per-query cost** (one OpenAI embedding round-trip + a local cross-encoder pass over ~30 candidates), not corpus size. The highest-leverage fix for latency at real traffic would be caching/batching the embedding call or a faster rerank model — not sharding the vector index further.

Pipeline wiring: `index_repo` end to end

`pipeline/index.py`

`index_repo()` composes everything above: walk the repo, route each file to its chunker, embed resulting chunks in batches of 64, upsert into the corpus's Qdrant collection. Each stage is wrapped in `timed_stage()`. Batching is intentional — embedding one chunk at a time means one network round-trip per chunk against an API that charges and rate-limits per call.

RESILIENCE IS A DESIGN DECISION, NOT AN AFTERTHOUGHT

A single unreadable/corrupt file is caught, logged as a warning, and skipped — indexing the rest of the repo continues. A Python file with a syntax error is caught one layer down inside `chunk_python_source` itself. Both are backed by real tests in `test_failure_modes.py` that simulate these conditions and assert on the behavior.

API layer: a factory app, and ingestion as a background job

`api/app.py``api/jobs.py``api/main.py`

`create_app(store, embedding_service, reranker, openai_client)` is a **factory function**, not a module-level global — tests inject an in-memory store and fake embedders instead of the real multi-gigabyte index and a real OpenAI client, letting the whole API test suite run in seconds with zero network calls.

Routes: `/health`; `/repos`; `/search` (raw hybrid retrieval, timed stage-by-stage for the UI's latency panel); `/ingest` + `/ingest/{job_id}`; `/explain` and `/explain/stream` (SSE).

WHY /INGEST RETURNS A JOB ID INSTEAD OF BLOCKING

Clone + chunk + embed for a real repo takes minutes. `JobStore.run()` spins the work onto a background thread and returns a job id immediately; the caller polls `GET /ingest/{job_id}`. Holding one HTTP connection open for a multi-minute request is fragile against proxy/load-balancer timeouts.

A KNOWN, STATED LIMITATION — NOT A HIDDEN ONE

`JobStore` is in-memory and single-process, matching today's single-machine deployment. A multi-worker deployment would need job state shared across processes (Redis, a DB row) — the code says this outright rather than presenting a finished, infinitely-scalable design.

Observability: one JSON line per stage, no exceptions

```
observability/timing.py
```

`timed_stage()` is a context manager: every indexing stage emits one structured JSON log line with its exact duration on completion, success or failure. If indexing is slow, grepping logs for `"stage": "embed"` tells you which stage, instead of guessing from a stopwatch on the whole request.

A REAL DEBUGGING FINDING BAKED INTO THE CONFIG

`configure_json_logging()` explicitly quiets `httpx / httpcore / urllib3 / huggingface_hub / filelock` to WARNING — found while debugging what looked like a hung multi-minute `/ingest` request that was actually dozens of INFO-level HTTP logs burying the pipeline's own progress lines.

The Streamlit UI: chat, raw search, and a client-side groundedness dashboard

streamlit_app/app.py

Three views. **Ask** is a chat interface streaming `/explain/stream`'s SSE events live — tool calls render as they happen and the answer types out token by token. **Raw search** bypasses the agent entirely, showing exactly what hybrid+rerank+RRF returns with per-stage millisecond timings.

THE EVALS VIEW MEASURES SOMETHING SPECIFIC AND SAYS SO

Every question is scored by citation-verification pass rate — whether each `file:line` cited actually exists. The UI is explicit this measures **groundedness, not correctness**: a wrong claim can still cite a real line, and nothing here re-judges the answer with another LLM call, since that would reintroduce, one layer up, the exact hallucination risk citation verification exists to catch.

Streamlit over a custom React frontend was a deliberate scope call: this project's value is retrieval quality and evaluation rigor, not UI polish.

Testing strategy: fakes by design, failure modes by intent

tests/

The fast suite (`pytest -q` , ~10s) uses fakes exclusively — a fake embedder returning fixed vectors, an in-memory Qdrant instance — made possible by the `Embedder` Protocol and the FastAPI factory pattern. A separate `slow` -marked suite downloads and runs the real cross-encoder. CI runs both plus the eval gate, on a 3.10/3.12 matrix, plus ruff and mypy.

TESTS THAT PROVE FAILURE IS LOUD AND CLEAN, NOT SILENT

`test_failure_modes.py` deliberately breaks five things: an unreachable Qdrant address fails fast, not hangs; a corrupted file is skipped with a warning, not an aborted repo; a crashing embedder raises rather than partially upserting; a wrong-dimension vector is rejected outright, with nothing committed for that failed batch.

Scaling past one machine

docs/scaling_design.md

Sharding by repository, not by language or a point-ID hash — a repo's chunks are almost always queried together, so co-locating them avoids turning every filtered query into a full cluster fan-out, and makes re-indexing/deletion single-shard operations.

Repo-level access control reuses the exact filter mechanism already built for the repo/language search filters — an allowed-repo list becomes one more condition in the same Qdrant filter, enforced at the database layer, not a post-fetch-then-discard step in application code.

Incremental re-indexing via GitHub push webhook re-chunks only added/modified files and scroll-deletes stale points before upserting fresh ones — safe specifically because of the deterministic point-ID decision made in the schema layer.

CHECKED AGAINST REAL NUMBERS, NOT LEFT AS PURE ARGUMENT

The load test sharpens this: sharding by repo is still right for write isolation, but the finding that query latency is dominated by fixed per-query cost (not corpus size) means sharding wouldn't fix query latency at real traffic — reducing or parallelizing the embedding call and rerank pass would.

Interview Q&A cheat sheet

Why does chunking code differently from prose matter?

A function is one unit of meaning; a fixed character window doesn't know where it starts or ends. I built two real structural chunkers — Python's own `ast`, and tree-sitter for JS/TS — plus a method-level split for classes, because a whole-class chunk was measured as the biggest ceiling on retrieval quality.

Walk me through what happens when reranking makes things worse.

On my own ablation, adding the cross-encoder alone dropped MRR from 1.000 to 0.803 — it's trained on prose ranking and biased toward README-shaped text over sparse-but-correct code. RRF fusion, blending the reranker's ordering with the pre-rerank order by rank rather than raw score, recovered it to 1.000. A measured fix for a measured failure, not a theoretical one.

How do you prevent the agent from hallucinating an answer?

Two layers: the system prompt requires every claim backed by a tool call made that turn, cited as file:line; and every citation gets checked by plain deterministic code against the real repo. That check can't itself hallucinate — it's not an LLM call. It's narrow on purpose: it proves the location is real, not that the cited lines prove the adjacent sentence, which would need semantic judgment and reintroduce the exact risk this exists to remove.

Tell me about something you tried that didn't work.

HyDE query expansion. Solid theory — generate a hypothetical answer, embed that instead of the raw question, close the vocabulary gap. Measured on my hard-eval set, it dropped MRR from 0.614 to 0.578, worse than not having it — because these questions are about this codebase's own idiosyncratic implementation, and a small model's "plausible-sounding" guess drifts away from the actual wording. It's still in the codebase, fully wired and tested, just behind an opt-in flag instead of the default.

Why an agent loop instead of just returning ranked chunks?

Two structural gaps pure retrieval can't close: it can't chase a reference across chunks, and a ranked list isn't an answer. The loop is one model calling three tools iteratively — the same pattern Claude Code uses — not multiple cooperating agents; no demonstrated win from splitting roles at this scale.

What would a real production deployment still need?

The background-job store is in-memory and single-process by design — fine for one machine, and the code says so rather than pretending otherwise. Multi-worker would need job state in Redis or a DB row. The scaling doc covers sharding by repo and access control reusing the existing metadata filter — designed and checked against load-test numbers, but not built, because the project's actual scale doesn't need it yet.

Isn't your eval set biased, since you wrote it yourself?

Yes, and I'd say that upfront. I mitigated it with two sets — a small, unambiguous 10-question set for cheap CI regression detection, and a harder 28-question paraphrased set for actually discriminating between changes — and by evaluating without filtering to the ground-truth item's known-correct repo, so recall reflects a real, repo-agnostic query. A larger, independently-authored set is the honest next step.

Glossary, in plain English

Embedding

A list of numbers representing a piece of text's meaning — similar meaning lands close together, measurably (cosine similarity).

tree-sitter

A general-purpose incremental parsing library covering dozens of languages — used here for JS/TS structural chunking.

Hybrid search

Running vector and keyword search in parallel and merging results, so both conceptual questions and exact identifiers work.

RRF

Reciprocal Rank Fusion — combines two rankings by $1/(k+\text{rank})$, summed. Lets a second opinion influence order without fully overriding a strong first-stage result.

Recall@k

Was the correct answer anywhere in the top k results? Binary per question, averaged.

Idempotent

Running an operation twice has the same effect as once — re-indexing an unchanged chunk overwrites the same point instead of duplicating it.

AST

Abstract Syntax Tree — a parser's structured, tree-shaped view of code, instead of plain text.

HNSW

Hierarchical Navigable Small World — the graph algorithm Qdrant uses to find close vectors fast.

Cross-encoder

A model that scores a (query, passage) pair jointly rather than comparing two independent vectors — more accurate, far more expensive per candidate.

HyDE

Hypothetical Document Embeddings — embed a generated hypothetical answer instead of the raw query.

MRR

$1 \div \text{rank}$ of the first correct result, averaged — rewards ranking the right answer near the top.

Ground truth

A hand-verified correct answer for a test question — here, the exact repo/file/symbol a question should retrieve.

Written directly from the current src/codeseek source tree, docs/, and CI config — every file path, constant, and number traces to a specific line in the repository.